

Disk Scheduling Under Mixed Workloads

Analyzing Scheduling Decisions Beyond Simple Request Queues

Prashant Jadav

DEPSTAR - CHARUSAT

What Is a Mixed Workload?

- A real system rarely generates one uniform type of disk request.
- A **mixed workload** may contain:
 - Random and sequential requests
 - Read and write operations
 - Interactive and background applications
 - Small and large I/O requests
 - High- and low-priority requests
 - Bursty and continuously arriving requests

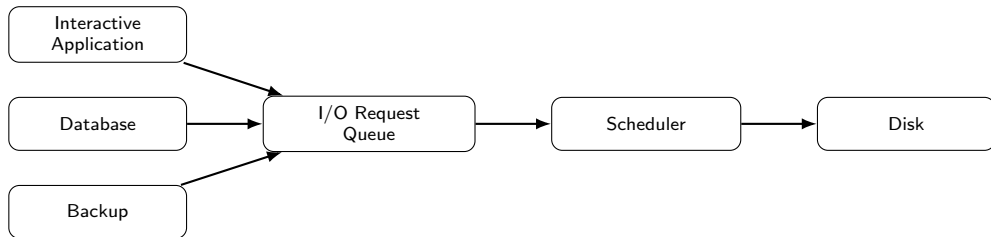
Challenge

A scheduling policy optimized for one workload characteristic may perform poorly when several characteristics occur simultaneously.

Example

A database may issue latency-sensitive reads while a backup process generates a large stream of background writes.

Why a Simple Queue Is Not Enough



- Requests may have different deadlines and importance.
- Request proximity is only one scheduling factor.
- The OS must balance **latency, throughput, fairness, and priority**.

Characteristics of I/O Requests

Characteristic	Possible values	Scheduling implication
Operation	Read / Write	Reads may be latency-sensitive; writes may be buffered
Size	Small / Large	Large requests can consume device bandwidth
Pattern	Sequential / Random	Sequential requests may benefit from locality
Priority	High / Normal / Low	High-priority work may require earlier service
Arrival	Periodic / Bursty / Continuous	Bursty queues can change optimal decisions
Application	Interactive / Batch / Background	Response-time requirements differ

Key idea

A scheduler should consider more than just the physical location of a request.

Performance Metrics for Mixed Workloads

- **Response time:** Time from request arrival until service begins/completes, depending on the definition used.
- **Waiting time:** Time spent waiting in the I/O queue.
- **Throughput:** Number or amount of I/O completed per unit time.
- **Fairness:** Whether different request classes receive reasonable service.
- **Tail latency:** High-percentile response time, such as P_{95} or P_{99} .
- **Bandwidth:** Amount of data transferred per unit time.

Important

Average latency alone can hide a serious problem: a small number of requests may experience extremely long delays.

Average Latency vs. Tail Latency

Average latency

$$L_{\text{avg}} = \frac{1}{n} \sum_{i=1}^n L_i$$

- Useful overall measure.
- Can hide outliers.

Tail latency

- P_{95} : 95% of requests complete within this latency.
- P_{99} : 99% complete within this latency.
- Important for interactive services.

Example

If most database reads complete quickly but a few requests wait behind a large batch of background writes, the average may look acceptable while the tail latency becomes unacceptable.

FCFS Under Mixed Workloads

First-Come, First-Served

- Preserves arrival order.
- Simple and easy to reason about.
- Provides a basic form of fairness by arrival order.
- Does not exploit locality or request priority.

Mixed-workload problem

A large sequential backup request can arrive before a latency-sensitive database read. FCFS may make the database request wait even when it is physically close to the current head position.

Trade-off

FCFS provides simple ordering but can produce poor utilization and response time when request characteristics differ.

SSTF Under Mixed Workloads

Shortest Seek Time First

- Selects the request closest to the current head position.
- Can reduce immediate seek distance.
- Useful when minimizing mechanical movement is important.

Mixed-workload issue

SSTF does not inherently understand application priority. A low-priority request that is physically close may be selected before a latency-sensitive request farther away.

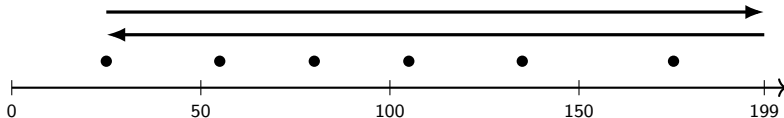
Additional risk

Continuous arrival of nearby requests can delay distant requests and increase tail latency.

SCAN Under Mixed Workloads

SCAN / Elevator scheduling

- Services requests while moving in one direction.
- Reverses direction and continues servicing.
- Provides structured movement across the disk.
- Reduces the starvation risk associated with purely local choices.



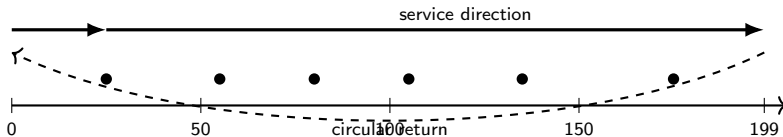
Mixed-workload benefit

SCAN gives requests across different disk regions regular opportunities for service, but does not by itself provide application-aware priority.

C-SCAN Under Mixed Workloads

Circular SCAN

- Services requests in one direction.
- Returns to the beginning and starts another sweep.
- Provides a more uniform service pattern across disk positions.



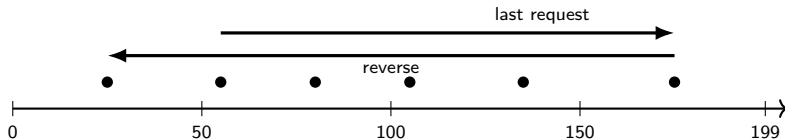
Mixed-workload benefit

Useful when predictable service across disk locations is more important than minimizing every individual seek.

LOOK Under Mixed Workloads

LOOK

- Similar to SCAN.
- Reverses direction at the last pending request.
- Avoids travelling to an unused physical boundary.
- Can reduce unnecessary movement compared with SCAN.



Mixed-workload consideration

LOOK can reduce unnecessary mechanical movement while retaining the structured behavior of an elevator-style scheduler.

Read vs. Write Requests

Reads

- Often directly affect application response time.
- Interactive workloads may require low read latency.
- Delayed reads can increase user-visible latency.

Writes

- May sometimes be buffered or reordered.
- Background writes can consume substantial bandwidth.
- Excessive write prioritization can delay reads.

Scheduling challenge

A scheduler may need to balance read latency against write throughput rather than simply treating every request identically.

Priority-Aware Scheduling

- Requests may be assigned different priorities.
- Example:
 - High: interactive database read
 - Medium: normal application I/O
 - Low: backup or batch processing
- The scheduler can consider priority together with device locality.

Important trade-off

Always serving high-priority requests first can starve low-priority work.

Fairness mechanism

Aging can gradually increase the effective priority of a request that has waited for a long time.

Priority + Seek Distance: A Balanced Decision

Suppose the queue contains:

Request	Cylinder	Priority
R_1	52	Low
R_2	85	High
R_3	120	Normal
R_4	57	High

Current head position:

50

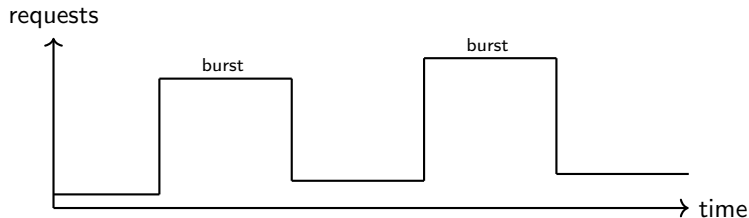
- Pure SSTF would favor R_1 at cylinder 52.
- A priority-aware scheduler may choose R_4 at 57 because it is high priority and still close.
- A strict priority scheduler may choose R_2 at 85 even though it is farther away.

Design question

How much should priority outweigh physical locality?

Bursty Workloads

- Requests may arrive in sudden bursts rather than at a constant rate.
- A burst can temporarily create a long request queue.
- The scheduling decision that was optimal before the burst may become suboptimal after new requests arrive.



Implication

Schedulers must operate dynamically because the queue contents and workload characteristics change over time.

Sequential + Random Access

Sequential workload

- Requests neighboring blocks.
- Benefits from locality.
- Read-ahead can be effective.
- Large transfers can improve throughput.

Random workload

- Requests scattered across the device.
- Seek movement can dominate HDD performance.
- Scheduling order becomes more important.

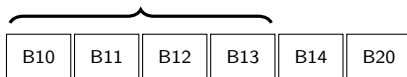
Mixed workload

A system may simultaneously perform sequential backup I/O and random database I/O. Treating both workloads identically can lead to poor latency or throughput.

Request Merging and Locality

- Nearby or adjacent requests may sometimes be combined into a larger request.
- Merging can reduce scheduling overhead and improve sequential transfer efficiency.
- It is particularly useful when multiple requests target neighboring blocks.

adjacent requests can be merged



Trade-off

Merging improves efficiency but must not introduce unacceptable delay for latency-sensitive requests.

Mixed Workload: A Scheduling Scenario

System

A university server is handling three workload classes:

Workload	Request pattern	Requirement
Student portal	Random reads	Low response latency
Database	Random read/write	Consistent performance and low tail latency
Nightly backup	Large sequential writes	High throughput, lower priority

Scheduling problem

A scheduler that maximizes sequential throughput for the backup may increase the portal's response time. A scheduler that always prioritizes portal reads may reduce backup throughput.

A Practical Scheduling Strategy

A mixed-workload scheduler can conceptually use several signals:

$$\text{Scheduling Score} = f(\text{priority, age, distance, deadline, request type})$$

- **Priority** – importance of the workload.
- **Age** – how long the request has waited.
- **Distance** – estimated device movement.
- **Deadline** – maximum acceptable delay.
- **Request type** – read/write or interactive/background.

Conceptual point

Real schedulers may use sophisticated policies; the important OS-design principle is to optimize for the actual workload rather than one metric alone.

Fairness Through Aging

- Priority scheduling can starve low-priority requests.
- **Aging** increases the effective priority of a request as its waiting time grows.
- This creates a balance between immediate priority and long-term fairness.

$$P_{\text{effective}} = P_{\text{base}} + f(\text{waiting time})$$

where $f(\cdot)$ increases as waiting time increases.

Example

A low-priority backup write that has waited for a very long time can gradually become eligible for service even while higher-priority interactive requests continue arriving.

Mixed Workloads on SSDs

- SSDs have no mechanical seek or rotational latency.
- Therefore, classical head-movement optimization is not the central issue.
- Important factors include:
 - Queue depth
 - Internal parallelism
 - Read/write interference
 - Garbage collection
 - Write amplification
 - Fairness and tail latency

Key point

The principles of workload-aware scheduling remain important even when the underlying device changes.

Comparing Classical Algorithms Under Mixed Workloads

Algorithm	Locality	Fairness tendency	Mixed-workload issue	Typical role
FCFS	Low	Good by arrival order	Ignores priority and locality	Simple baseline
SSTF	High	Lower	Can starve distant requests	Seek reduction
SCAN	Good	Good	No application priority	Balanced HDD scheduling
C-SCAN	Good	More uniform	Extra circular movement	Uniform service
LOOK	Good	Good	Still needs workload awareness	Efficient elevator behavior

Interpretation

These are workload tendencies, not universal performance guarantees.

Worked Analytical Example

Given

Current head position: 50

Pending requests:

$R_1(48, \text{low})$, $R_2(52, \text{high})$, $R_3(90, \text{normal})$, $R_4(150, \text{low})$

- ① **SSTF**: nearest request is R_1 at 48.
- ② **Priority-aware**: R_2 may be selected because it is high priority and only two cylinders away.
- ③ **Strict priority**: R_2 is selected regardless of distance.
- ④ **Fair scheduler**: may consider waiting time in addition to priority and distance.

Question

Which metric changes the decision: **distance, priority, or waiting time**? A practical scheduler may consider all three.

System-Level Design Trade-offs

- Minimizing seek distance does not necessarily minimize application response time.
- Maximizing throughput does not necessarily minimize tail latency.
- Strict priority can improve urgent-request latency but may cause starvation.
- Fairness can improve predictability but may reduce peak throughput.
- Request merging can improve efficiency but may delay an urgent request.
- Background I/O should generally not overwhelm latency-sensitive foreground work.

OS design principle

Optimize the workload's actual objective, not merely one local metric.

How to Analyze a Mixed-Workload Problem

- 1 Identify the workload classes.
- 2 Determine whether requests are reads or writes.
- 3 Identify sequential vs. random access.
- 4 Determine priority and latency requirements.
- 5 Identify burstiness and request-arrival behavior.
- 6 Choose measurable objectives:
 - Average latency
 - Tail latency
 - Throughput
 - Fairness
- 7 Evaluate the scheduling policy under the complete workload.

Do not conclude from one request sequence alone

A scheduling algorithm should be evaluated using representative workload patterns.

Quick Check

- ① What makes a workload “mixed”?
- ② Why can average latency hide a scheduling problem?
- ③ Why may SSTF be unsuitable for strict fairness requirements?
- ④ How can priority scheduling affect background workloads?
- ⑤ What is the purpose of aging?
- ⑥ Why can request merging improve performance?
- ⑦ Why are classical seek-based algorithms less central for SSDs?

Higher-Order Discussion Problem

Scenario

A university server simultaneously handles:

- Interactive database reads
- Student portal requests
- Large backup writes

The backup generates mostly sequential I/O, while portal and database traffic is mostly random.

- 1 Identify the conflicting performance objectives.
- 2 Explain why FCFS may produce poor response time.
- 3 Explain how SSTF could affect fairness.
- 4 Explain how SCAN/LOOK can provide more structured service.
- 5 Propose the metrics that should be monitored.
- 6 Explain how aging or priority could be incorporated.

Key Takeaways

- Real systems commonly have **mixed I/O workloads**.
- Requests differ in priority, size, access pattern, operation type, and latency requirement.
- Average latency alone is insufficient; **tail latency and fairness** can be important.
- FCFS is simple, SSTF exploits locality, and SCAN/C-SCAN/LOOK provide structured movement.
- Priority-aware scheduling can protect latency-sensitive work but may starve background work.
- Aging can improve fairness over time.
- Request merging and workload-aware policies can improve efficiency.
- On SSDs, scheduling concerns shift away from mechanical seek distance toward parallelism, interference, fairness, and tail latency.

Good I/O scheduling balances locality, latency, throughput, priority, and fairness.